

tion is shown in Fig. 3. Upon any key press, the ASCII equivalent of the pressed key is identified and is returned. Hence, in the testing of the circuit given in Fig. 1, the lower nibble alone is sufficient as the LEDs indicate the pressed digit.

As an example, if the user presses key-'1', the ASCII of '1', which is 0x31, i.e., 0110 0001, is shown on the LEDs. In the same way, all the keys of the keypad may be tested by observing their equivalent binary outputs on LEDs.

Let us now look at the approach of converting the above polled-loop driver by binding the interrupt and demonstrate the process of context switching from the application to interrupt service routine (ISR), and vice-versa.

Circuit diagram for interrupt-driven keypad

There are two external interrupt pins /INT0 and /INT1 of AT89S52, which are available as alternate-pin functions of the P3.2 and P3.3 GPIOs, respectively. Among these, let us consider usage of /INT0 pin, the P3.2 pin. A three-input AND gate 4073 IC is used such that its output must be connected to /INT0 (P3.2), while the inputs are connected to the three columns C1, C2, and C3, as shown in Fig. 4.

Implementation of interrupt-driven driver

For achieving the key input, as the columns were locked to interrupt pin, any key pressed will result into a change of column signal and thereby generate an interrupt at /INT0 pin. We must enable the interrupt /INT0 pin by configuring the IE register. Moreover, we must configure it as an edge-triggered type of interrupt us-

ing TCON register in main application.

The basic configuration in the code for Project3 is shown in Fig. 5. The MCU's C-ISR function 'Keypad_ISR' is implemented as indicated in Fig. 6.

Some very important observations of the implemented ISR are:

1. The interrupt must be configured as edge triggered. This is achieved in AT89S52 by configuring TCON register. Otherwise, even for a single press the interrupt will keep on running as if multiple events are occurring.

2. An interrupt service routine function must not return any value. Hence, we use the concept of global shared resource, which in our case will be the global variable. In the implementation, the variable 'Key' will be acting as a shared resource. Notice that this shared resource is accessible not only in the main but also in the ISR.

3. In the implementation given here an LED is intentionally connected to the unused GPIO P1.7 pin of AT89S52, which will toggle upon the key press due to line-3 of Fig. 6.

4. Whenever a key is pressed by the user, the corresponding column status in the keypad will be changed to 1. As a result, the interrupt will be triggered, and the application will be paused as it enters into /INT0 ISR. This process is called hardware context switching. Notice that during this time, the system stack will be used. (Bank-1 in RAM)

```
1. static unsigned char Key;
2. void main(void)
3. {
4.     int i=0;
5.     P2=0;
6.     R1=R2=R3=R4=0;
7.     C1=C2=C3=1;
8.     IE = 0x81; //Enable INT0 interrupt.
9.     TCON = 0x01; //Edge-triggered interrupt.
10. Key=0;
11.     while(1); //Actual Application can be implemented here.
12. }
```

Fig. 5: Device initialisation routine in the main code for interrupt driven keypad driver

```
1. void Keypad_ISR(void) interrupt 0
2. {
3.     ISR_LED=~ISR_LED; //LED used for testing ISR
4.     R1=0;R2=R3=R4=1;
5.     if(C1==0)
6.     {
7.         Key='1';
8.         return;
9.     }
10.    else if(C2==0)
11.    {
12.        Key='2';
13.        return;
14.    }
15.    else if(C3==0)
16.    {
17.        Key='3';
18.        return;
19.    }
20.
21.    R2=0;R1=R3=R4=1;
22.    if(C1==0)
23.    {
24.        Key='4';
25.        return;
26.    }
27.    else if(C2==0)
28.    {
29.        Key='5';
30.        return;
31.    }
32.    else if(C3==0)
33.    {
34.        Key='6';
35.        return;
36.    }
37.
38.    R3=0;R1=R2=R4=1;
39.
40.    if(C1==0)
41.    {
42.        Key='7';
43.        return;
44.    }
45.    else if(C2==0)
46.    {
47.        Key='8';
48.        return;
49.    }
50.    else if(C3==0)
51.    {
52.        Key='9';
53.        return;
54.    }
55.
56.    R4=0;R1=R2=R3=1;
57.    if(C1==0)
58.    {
59.        Key='*';
60.        return;
61.    }
62.    else if(C2==0)
63.    {
64.        Key='0';
65.        return;
66.    }
67.    else if(C3==0)
68.    {
69.        Key='#';
70.        return;
71.    }
72. }
```

Fig. 6: Keypad C ISR for AT89S52 MCU